

Understanding Ledger's Software Suite

A guide to the libraries, repos and tools, built around the questions in your notes: which library to use, which ones cover UTXO chains, why the DMK exists, and how many options you really have.

The short version

Which library should I use? For anything new that talks to a Ledger device, use the **Device Management Kit (DMK)**: one core package, one transport for your platform, and one signer kit per blockchain. The older **ledgerjs** packages (`hw-transport-*`, `hw-app-*`) are deprecated for new work. If your code runs *inside* Ledger Wallet as a Live App, you use the **Wallet API** instead and never touch the device directly.

Which ones support all UTXO chains? None of the DMK signers does.

`device-signer-kit-bitcoin` covers Bitcoin, `device-signer-kit-zcash` covers Zcash. The broadest UTXO coverage still lives in the legacy `@ledgerhq/hw-app-btc`, which handles Bitcoin plus several Bitcoin-derived altcoins. Section 6 has the details.

Why was the DMK created? The old stack only gave you a pipe for raw bytes (APDUs). Everything else (is the device locked, is the right app open, did the user approve, what went wrong) was left to each integrator, and Ledger Live reimplemented it privately. The DMK moves all of that into one shared, multi-platform kit. Section 3.

How many options do I have? Fewer than the list suggests. Your notes name 2 core packages, 9 transport-related packages (6 actual transports), 3 trusted-app kits and 12 signers, but any single integration uses about 3 of them: 1 core + 1 transport + 1 signer. Section 5.

1. The big picture: three worlds

Most of the confusion in the notes comes from the fact that "Ledger software" spans three very different things that live in different repositories and are written in different languages. Keeping them apart makes everything else easier.

- **The device** runs Ledger's OS (called **BOLOS**, which is what `GetAppAndVersion` reports when you are on the dashboard) plus small per-chain **device apps** (the Ethereum app, the Bitcoin app, and so on). These are written in C or Rust and live in repos such as `app-ethereum`, `app-bitcoin`, `ledger-secure-sdk` and `ledger-device-rust-sdk`.
- **Host libraries** run on your computer or phone and send commands (APDUs) to the device. This is where the DMK and the legacy `ledgerjs` packages sit, and it is the part your question "which library should I use?" is about.
- **Products** are things built on the host libraries: Ledger Wallet (formerly Ledger Live), third-party wallets and dApps, and command-line tools.

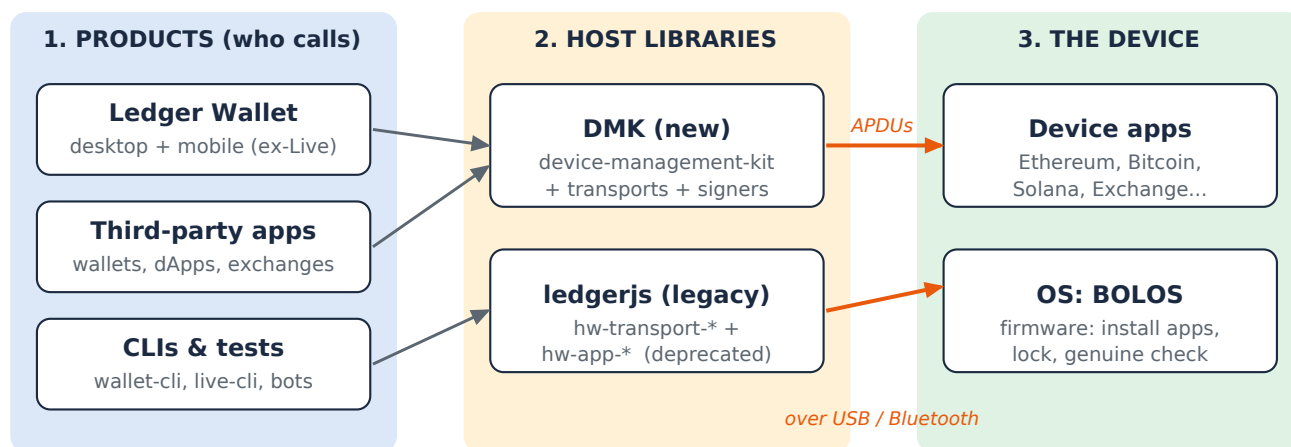


Figure 1. Everything Ledger ships falls into one of these three columns.

A useful mental model: the device app is a tiny server, the APDU is its wire protocol, and a signer kit is the typed client for that server. The DMK is the HTTP-client equivalent that every signer kit shares.

2. How a request travels: the layer stack

Your notes describe DMK as "like the application layer" and Transport as "like the link layer". That analogy is accurate. Here is the full stack for one operation, such as signing an Ethereum transaction:

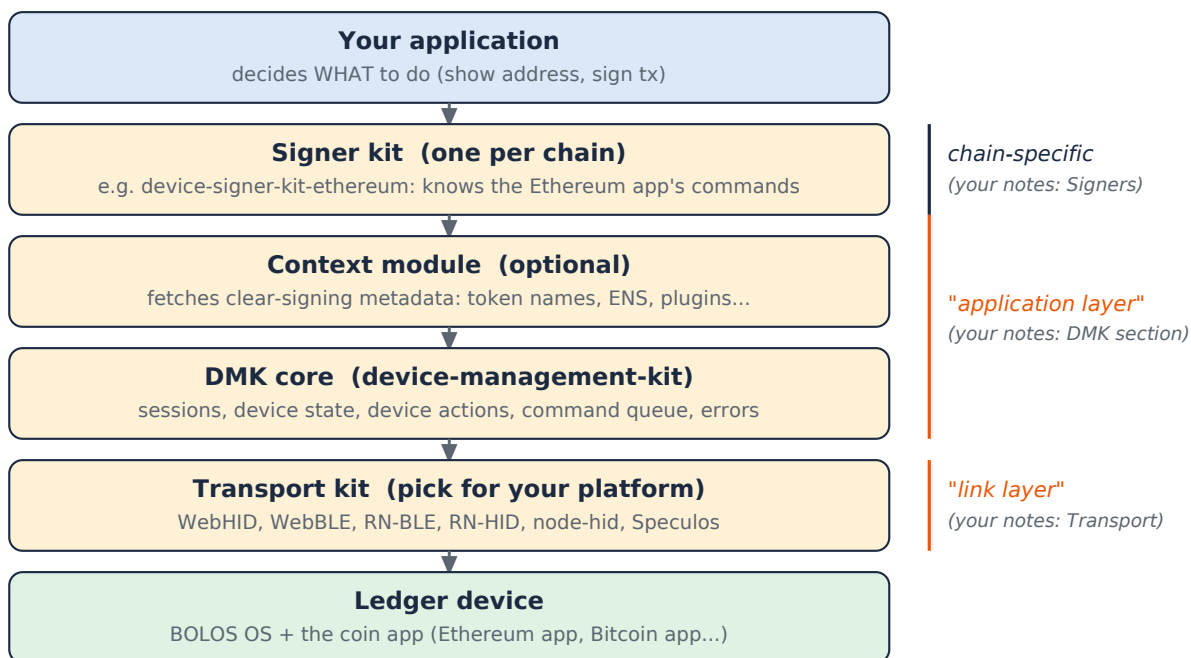


Figure 2. The DMK stack. Your notes' four section headings map onto these layers.

What each layer is responsible for

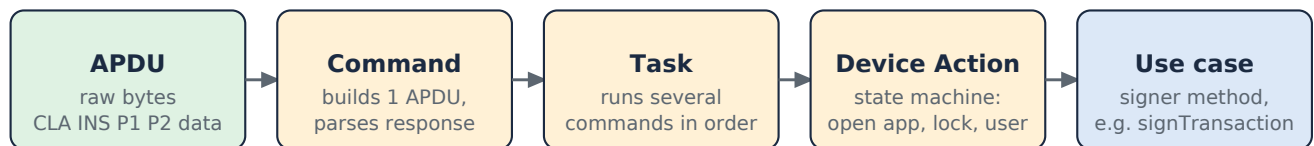
- **Transport** only knows how to move bytes over one physical link: USB in a browser (WebHID), USB in Node (node-hid), Bluetooth in a browser or React Native, or an emulator over HTTP

(Speculos). It doesn't know what the bytes mean.

- **DMK core** manages *sessions*. Once connected, you can observe the device's state (ready, busy, locked, disconnected), its OS version, battery and which app is open. It serializes commands so two requests never collide, and ships OS-level actions such as Open App, Go To Dashboard, List Apps, Install/Uninstall App and Genuine Check.
- **Context module** is a helper used by signers for *clear signing*: before the device shows a transaction, it fetches signed metadata (token tickers, NFT names, ENS domains, plugin info, transaction-check verdicts) from Ledger's backend and loads it onto the device, so the screen says "Send 10 USDT to vitalik.eth" instead of raw hex.
- **Signer kit** knows one device app's protocol and exposes friendly methods such as `getAddress`, `signTransaction` and `signMessage`.

Inside the DMK: commands, tasks, device actions

Reading the READMEs, you'll keep meeting the same four nouns. They nest:



Each level wraps the one to its left. You normally only touch the rightmost one; it returns { observable, cancel } and the observable emits DeviceActionState updates.

Figure 3. The DMK's internal building blocks.

The important practical consequence: a signer method doesn't return a single promise. It returns an observable that emits states: `NotStarted`, `Pending`, `Stopped`, `Completed` or `Error`. While pending, it tells your UI what the user must do on the device through `requiredUserInteraction` (`UnlockDevice`, `ConfirmOpenApp`, `SignTransaction`, `VerifyAddress`...). That is how a wallet knows when to show "Please unlock your Ledger".

3. Two generations: ledgerjs vs. the DMK

The line at the bottom of your notes, "hw-app-* are the old versions", is the key fact. There are two generations of host libraries, and both still exist in the wild.

	Legacy: ledgerjs	Current: DMK
Packages	<code>@ledgerhq/hw-transport-*</code> and <code>@ledgerhq/hw-app-*</code>	<code>@ledgerhq/device-management-kit</code> , <code>device-transport-kit-*</code> , <code>device-signer-kit-*</code>
Repo	ledger-live monorepo, <code>libs/ledgerjs/</code> (the old <code>ledgerjs</code> repo is archived)	<code>device-sdk-ts</code> (separate repo, published to npm)
Core abstraction	A <code>Transport</code> with <code>exchange(apdu)</code> . That's basically it.	A session with observable state, plus device actions that orchestrate many commands
App open / lock / user prompts	Your problem. Ledger Live built its own (<code>connectApp</code> , <code>withDevice</code>) in live-common.	Built in. Emits <code>UnlockDevice</code> , <code>ConfirmOpenApp</code> , etc. as intermediate states.
Results & errors	Promises; errors thrown as status-word exceptions (<code>@ledgerhq/errors</code> , now deprecated and frozen)	Typed results and typed error classes per action

	Legacy: ledgerjs	Current: DMK
Clear signing	Per library, e.g. hw-app-eth's resolution services	Shared context module used by signers
Status	Deprecated for new integrations; remaining transports in the repo are dev/test only	Recommended path; Ledger Wallet itself is migrating to it

Why the DMK was created

The material you shared states the *what* plainly: the ledgerjs README now says the DMK is "the recommended way to communicate with a Ledger device" and that new integrations should use it, and the DMK README describes it as handling discovery, connection, device management with "comprehensive status tracking and error handling", OS and app interaction, and app installation, across web, Android and iOS. The *why* is easy to infer by comparing the two stacks:

- **The old interface was too thin.** `Transport` gave you `open`, `exchange` and `close`. Every integrator then had to figure out whether the device was locked, which app was open, how to open the right one, and how to map cryptic status words to messages. That logic was duplicated in every wallet.
- **Ledger's own best logic was trapped in Ledger Live.** The robust orchestration lived in `ledger-live-common` as private device actions. Third parties couldn't reuse it. The DMK makes that orchestration a public, shared product.
- **One API across platforms.** Desktop, browser, React Native and Node all used slightly different transports and conventions. In the DMK, the transport is a plug-in chosen at build time and everything above it is identical.
- **Concurrency and state.** The DMK queues commands per session and tracks device state continuously, which avoids the classic "two exchanges at once" failures.
- **Better UX hooks and clear signing.** Observable states with explicit user-interaction prompts, plus a shared context module, make it practical for any wallet to offer clear signing rather than blind signing.

The first bullet in this list is directly visible in the READMEs; the rest are reasonable inferences from how the two APIs differ. If you need Ledger's official rationale, check the Ledger Developer Portal's DMK pages.

4. The DMK package catalog, annotated

This is your notes' list with a plain-language description of each entry. Where the material you shared contains no README for a package, I've said so rather than guess.

Core

Package	What it is	Who needs it
<code>device-management-kit</code>	The kit itself: builder, sessions, discovery, commands, OS-level device actions.	Everyone using DMK
<code>dmk-ledger-wallet</code>	Extra device actions only Ledger Wallet needs: custom lock screen upload/download/remove, backup. The README says other apps don't need it.	Ledger Wallet only

Transports (pick by platform)

Package	Link	Runs in
<code>device-transport-kit-web-hid</code>	USB	Browsers with WebHID (Chromium-based)
<code>device-transport-kit-web-ble</code>	Bluetooth	Browsers with Web Bluetooth
<code>device-transport-kit-react-native-ble</code>	Bluetooth	React Native, iOS 11+ and Android 6+
<code>device-transport-kit-react-native-hid</code>	USB	React Native, Android only
<code>device-transport-kit-node-hid</code>	USB	Node.js (tested on v20): scripts, CLIs, servers
<code>device-transport-kit-speculos</code>	HTTP	Talks to the Speculos emulator; Node or browser
<code>speculos-device-controller</code>	(helper)	Not a transport. Presses buttons / taps the emulated screen in tests.
<code>device-transport-kit-mockserver</code> + <code>device-mockserver-client</code>	(mock)	Fake device for automated tests

Trusted-app kits

Package	What it is
<code>device-trusted-app-kit-ledger-keyring-protocol</code>	Ledger Key Ring Protocol (LKR), the device side of Ledger Sync: building a "trustchain" of members and deriving shared encryption keys. Still 0.x.
<code>context-module</code>	Clear-signing metadata fetcher used by signer kits (it sits under <code>packages/signer/</code> in the repo even though your notes file it here). Now multichain: you call <code>setChain()</code> when building it.
<code>device-contacts-kit</code>	Listed in your notes but not documented in the material you shared. Ledger Wallet has a Contacts feature (<code>@features/flow-contacts</code>), so it's likely related.

Signers (one per chain / device app)

Signer kit	Version	Main methods (from the READMEs)
Ethereum	1.16	<code>getAddress</code> , <code>signTransaction</code> (clear signing), <code>signMessage</code> , <code>signTypedData</code> (EIP-712), <code>signDelegationAuthorization</code> (EIP-7702), <code>verifySafeAddress</code>
Bitcoin	1.3	<code>getExtendedPublicKey</code> , <code>getMasterFingerprint</code> , <code>getWalletAddress</code> , <code>signMessage</code> , <code>signPsbt</code> , <code>signTransaction</code> , <code>registerWallet</code> (multisig / wallet policies)
Solana	1.10	<code>getAddress</code> , <code>signTransaction</code> (clear signing, delayed signing), <code>signMessage</code> (off-chain V0/V1/Legacy/Raw), <code>getAppConfiguration</code>
Cosmos	1.0	<code>getAddress</code> (with chain prefix, e.g. cosmos / osmosis), <code>signTransaction</code> , <code>getAppConfig</code>
Hyperliquid	1.1	<code>signActions</code> (orders, cancels...)
Zcash	0.4	<code>getAddress</code> , <code>getFullViewingKey</code> , <code>signTransaction</code> (transparent), <code>signPcztTransaction</code> (Orchard shielded), <code>signMessage</code> , <code>getTrustedInput</code>
Concordium	0.4	<code>getPublicKey</code> , <code>signTransaction</code> , <code>signCredentialDeploymentTransaction</code> , <code>verifyAddress</code>
Aleo	0.4	<code>getAddress</code> , <code>getViewKey</code> , <code>signTransaction</code>
Polkadot	0.1	<code>getAddress</code> , <code>signTransaction</code> (with runtime metadata)

Signer kit	Version	Main methods (from the READMEs)
Tron	0.1	getAddress, signTransaction, signPersonalMessage, getAppConfiguration
ICP, XRP	n/a	Listed in your notes; no README in the material shared

Version numbers are a decent maturity signal: 1.x packages are established, while 0.x packages are newer and more likely to change their API.

5. Choosing: how many options do you really have?

The catalog looks long, but it's a menu, not a list of alternatives. For any one project you make three small choices:

Choice	How many	How to decide
Core	1	Always device-management-kit . Add dmk-ledger-wallet only if you are Ledger Wallet.
Transport	1 or 2 of 6	Where your code runs: browser (web-hid and/or web-ble), React Native (rn-ble, plus rn-hid on Android), Node (node-hid). Add Speculos or mockserver for tests.
Signer	1 per chain	One per blockchain you support. If your chain isn't in the 12, see the fallback below.

Decision guide by goal

I want to...	Use
Add Ledger support to a new web wallet or dApp	DMK + web-hid (and/or web-ble) + signer kit(s) for your chains
Add Ledger support to a React Native app	DMK + react-native-ble (+ react-native-hid for Android USB) + signer kit(s)
Write a Node script, backend tool, CLI or Electron main-process code	DMK + node-hid + signer kit(s)
Test without real hardware	Speculos transport + speculos-device-controller, or the mockserver pair
Sign for a chain with no DMK signer kit	The legacy hw-app-* for that chain. Ledger Wallet runs these on top of a DMK session through an adapter called DmkCompatTransport (in @ledgerhq/live-dmk-shared).
Build an app that runs inside Ledger Wallet (a Live App)	Wallet API client and modules (exchange, feature-flag, deeplink...). For EVM dApps, @ledgerhq/ethereum-provider gives you an EIP-1193 window.ethereum . No direct device access.
Use my own Ledger from a terminal	@ledgerhq/wallet-cli (Bitcoin, Ethereum, Solana: balances, send, receive, swap, earn). The older @ledgerhq/live-cli is now Speculos-only CI plumbing.
Write code that runs on the device itself	Section 9: app-boilerplate-rust, speculos, ragger...
Contribute to Ledger Wallet	The ledger-live monorepo. Section 8.

What the code looks like

Simplified from the READMEs (check each package's README for exact signatures; the examples vary slightly between packages). Old style first, then DMK:

```
// Legacy ledgerjs: open a transport, wrap it in an app class, await a promise
const transport = await TransportWebHID.create();
const eth = new Eth(transport);
const { address } = await eth.getAddress("44'/60'/0'/0/0");

// DMK: build once, discover + connect, then use a signer kit
const dmk = new DeviceManagementKitBuilder()
  .addTransport(webHidTransportFactory)
  .build();

dmk.startDiscovering().subscribe(async (device) => {
  const sessionId = await dmk.connect({ deviceId: device.id });
  const signer = new SignerEthBuilder({ dmk, sessionId }).build();

  const { observable, cancel } = signer.signTransaction("44'/60'/0'/0/0", txBytes);
  observable.subscribe((state) => {
    if (state.status === DeviceActionStatus.Pending)
      showPrompt(state.intermediateValue.requiredUserInteraction); // e.g. UnlockDevice
    if (state.status === DeviceActionStatus.Completed) use(state.output); // { r, s, v }
    if (state.status === DeviceActionStatus.Error) handle(state.error);
  });
});
```

6. UTXO chains: what actually covers them

UTXO chains are Bitcoin and its descendants (Litecoin, Dogecoin, Bitcoin Cash, Zcash's transparent pool...), plus a few chains with a different UTXO design such as Cardano and Kasper. Coverage is uneven, and the migration table in your notes hides a gotcha here.

Library	UTXO coverage	Notes
<code>device-signer-kit-bitcoin</code> (DMK)	Bitcoin	Speaks the modern Bitcoin app protocol (PSBT v0/v2, wallet policies, multisig, taproot, MuSig2 nonces/signatures). The README describes Bitcoin only, with no altcoin options.
<code>device-signer-kit-zcash</code> (DMK)	Zcash	Transparent payments (trusted-input flow compatible with Ledger Wallet) and Orchard shielded signing via PCZT.
<code>@ledgerhq/hw-app-btc</code> (legacy)	Bitcoin + several altcoins	README: "Also supports many altcoins." Takes a currency parameter and additional s flags such as abc (Bitcoin Cash), gold (Bitcoin Gold), decred , zcash/sapling , and a cashaddr address format. Contains both BtcNew (Bitcoin app 2.1.0+) and BtcOld (older protocol).
<code>@ledgerhq/coin-bitcoin</code> (Ledger Wallet)	The whole bitcoin family	Not a device library: the coin module inside Ledger Wallet that syncs accounts and builds transactions for every currency registered with BitcoinLikeInfo . It drives the device through a signer.
<code>hw-app-kaspa</code> , Cardano libraries	Kaspa, Cardano	Separate, non-Bitcoin protocols. Kaspa's lib is community-maintained; Cardano's is published by the Cardano Foundation.

Bottom line for UTXO

No single library covers every UTXO chain. The legacy `hw-app-btc` has the widest reach because it was designed around a family of Bitcoin-derived device apps. Your notes' migration table says `hw-app-btc` → `device-signer-kit-bitcoin`, but as far as the READMEs show, that swap is only like-for-like for Bitcoin itself. If you support Litecoin, Dogecoin or Bitcoin Cash, confirm the DMK signer handles them before migrating; otherwise keep `hw-app-btc` for those chains, ideally running over a DMK session.

To get the definitive list of Bitcoin-family currencies Ledger Wallet supports, look at the currency registry in the monorepo (`@domain/entity-currency-crypto` or the legacy `@ledgerhq/cryptoassets`) and filter on `bitcoinLikeInfo`. That isn't in the material you shared, so I can't list them here.

Two related helpers in the monorepo: `@ledgerhq/psbtv2` (a PSBT v2 implementation, BIP370, convertible from v0) and the Bitcoin coin-tester, which runs deterministic end-to-end scenarios against a local chain.

7. Migrating from `hw-app-*` to the DMK

Your notes include the official mapping:

If you use...	Migrate to...	Watch out for
<code>hw-app-eth</code>	<code>device-signer-kit-ethereum</code>	Mature (1.16). Clear signing now goes through the context module, which needs an origin token.
<code>hw-app-btc</code>	<code>device-signer-kit-bitcoin</code>	Bitcoin only, as far as the docs show. API is PSBT-centric rather than the old <code>createPaymentTransaction</code> .
<code>hw-app-solana</code>	<code>device-signer-kit-solana</code>	Mature (1.10). Ledger Wallet's own Solana signer still uses the legacy lib.
<code>hw-transport-webhid</code>	<code>device-transport-kit-web-hid</code>	Transport becomes a builder plug-in, not something you open yourself.
<code>hw-transport-web-ble</code>	<code>device-transport-kit-web-ble</code>	
<code>hw-transport-node-hid</code>	<code>device-transport-kit-node-hid</code>	

Migrating incrementally

You don't have to migrate everything at once. Ledger Wallet shows the pattern: connect through the DMK, then wrap the session in `DmkCompatTransport`, a minimal object that looks like an old `Transport`. Legacy `hw-app-*` classes run unchanged on top of it, so you can move transport and session handling first and swap signers chain by chain later. Only `exchange` is bridged; the DMK owns the session, so you must not call `close()` on the adapter.

8. The ledger-live monorepo

Repo map: what's in your notes' table

Repo	Status	What it is
ledger-live	Active	The monorepo: Ledger Wallet Desktop (Electron), Mobile (React Native), wallet-cli, and 150+ npm libraries including the legacy ledgerjs packages and all coin modules.
ledger-live-desktop, -mobile, -common, ledgerjs	Archived 2022-06-08	All archived the same day: they were merged into the ledger-live monorepo.
lib-ledger-core	Archived 2025	The old C++ wallet core. Coin logic has been rewritten in JS/TS coin modules (the Tezos module README still mentions this rework).
ledger-wallet-chrome	Archived 2020	The original Chrome app wallet, long retired.
device-sdk-ts	Active	Home of the DMK, transports, signers, context module and devtools.
app-ethereum, app-bitcoin, app-exchange	Active	Device apps. app-exchange is the on-device swap/sell app; hw-app-exchange is its host library.
ledger-secure-sdk, ledger-device-rust-sdk	Active	SDKs for writing device apps in C and Rust.
speculos	Active	Device emulator (Nano S+, Nano X, Stax, Flex, Nano Gen5).
lumen	Active	Ledger's design system (@ledgerhq/lumen-ui-react / -rnative appear in the monorepo).
ledger-button-public, vanadium, ledger-app-database	Active	Not described in the material you shared.

"Why isn't everything here?"

That question in your notes has four answers:

- **Different languages and release cycles.** Device firmware and apps are C/Rust, built with special toolchains and shipped to devices. They can't sensibly live in a TypeScript app monorepo.
- **The DMK is a public SDK.** It lives in device-sdk-ts so third parties can depend on it, with its own versioning and changelogs. Ledger Wallet consumes it from npm like anyone else.
- **History.** Several repos were folded into ledger-live in June 2022 and archived, which is why the commit table shows so many dead repos.
- **The internal layout is mid-migration.** Inside the monorepo, the DDD folders (domain/, features/, shared/) are new. The domain README says it is "still maturing and is not yet the default home for new code". New shared code goes into a new libs/* package, and ledger-live-common is in maintenance mode, being "progressively dismantled". That's what your note "the repo is only partially using this architecture" refers to.

The DDD layout and dependency direction

Folder	Your notes' share	Contains
apps/	~20%	Platform entry points, routing, screens (desktop, mobile, CLI)
features/flow/	~60% (features total)	User-facing UI shared across apps (<code>.web.tsx</code> / <code>.native.tsx</code> variants)
features/platform/		Non-visual glue: hooks, selectors, feature flags, currency loading
domain/	~10%	Business objects: Zod schemas, Redux slices (<code>entity/</code>) and RTK Query APIs (<code>api/</code>)
shared/	~10%	Domain-agnostic primitives (feature flags, schema primitives)
libs/		Everything else, including the legacy ledgerjs <code>hw-app-*</code> / <code>hw-transport-*</code> (deprecated)

Imports may only point downward: `apps` → `features/flow` → `features/platform` → `domain` → `shared`. Lower layers never import higher ones, so domain code stays reusable and testable.

How Ledger Wallet itself talks to the device

This is useful because it shows the recommended architecture in a real product, and because Ledger Wallet is visibly partway through its own migration.

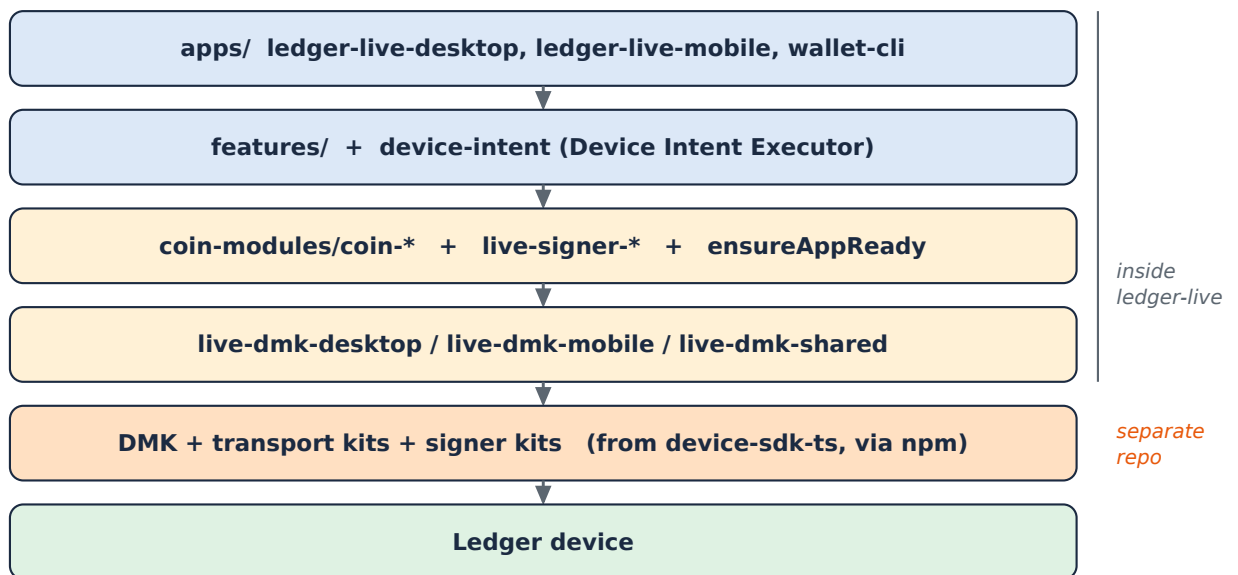


Figure 4. Ledger Wallet's device stack. Everything above the orange band lives in ledger-live.

- `live-dmk-desktop` / `live-dmk-mobile` wire DMK transports into each app (USB on desktop; Bluetooth and USB-C on mobile). `live-dmk-shared` holds common discovery, logging, blind-signing reporting and `DmkCompatTransport`.
- `device-intent` provides the **Device Intent Executor**, a React component that handles connect → prepare device (open/install the app via `ensureAppReady`) → run a job. Its README says it is "designed to replace" the legacy `DeviceAction` / `withDevice` plumbing.

- `live-signer-*` packages are Ledger Wallet's per-chain signers. Some wrap DMK signer kits, others still wrap legacy `hw-app-*`:

live-signer	Backed by
concordium, aleo, hyperliquid, zcash	DMK signer kits
evm	DMK or legacy hw-app-eth
cosmos	DMK signer kit alongside legacy hw-app-cosmos and ledger-cosmos-js
solana, canton	Legacy hw-app-solana / hw-app-canton
celo	Its own wrapper around the Celo app protocol (with a LegacySignerCelo)

Takeaway: if Ledger's own wallet still mixes both generations, it's normal for you to do the same during a transition.

9. The other half of your notes: device app development

The "Device App Development" list is about writing code that runs *on* the Ledger, not about talking to it. In context:

Tool	Role
<code>app-boilerplate-rust</code>	Forkable Rust skeleton. Your notes say it's mandatory for new apps.
<code>app-boilerplate</code>	Forkable C skeleton, for existing C projects
<code>app-plugin-boilerplate</code>	Skeleton for Ethereum-app plugins (how a protocol gets custom clear-signing screens inside the Ethereum app)
<code>ledger-app-ai-instructions</code>	Reusable CLAUDE.md / Copilot instruction files for AI-assisted app development
<code>ledger-dev-tools</code> (VS Code)	One-click build, test and load from the IDE
<code>speculos</code>	The emulator. Shared with host-side testing, which is why it appears in both halves of your notes.
<code>ragger</code>	pytest-based functional test framework for device apps
<code>ledger-app-workflows</code>	Reusable GitHub Actions workflows
<code>ledger-app-builder</code>	Docker images with the toolchain (the READMEs build apps with <code>ledger-app-dev-tools</code>)

The two halves meet at the APDU: a device app defines its commands, and a signer kit (or legacy `hw-app-*`) is the host-side client for exactly those commands. A new chain therefore usually needs both a device app and a signer kit.

10. Glossary

Term	Meaning
APDU	Application Protocol Data Unit: the byte-level command/response format (from smart cards) used for all host-device communication. Shape: CLA, INS, P1, P2, data; responses end with a status word, 0x9000 = success.
BOLOS	Ledger's device OS. When no app is open you're on the "dashboard", which reports itself as BOLOS.
Transport	The code that moves APDUs over one physical link (USB/HID, Bluetooth, HTTP to an emulator).
Session	DMK's handle on a connected device. All later calls take a <code>sessionId</code> .
Device action	A DMK state machine that orchestrates several commands and user prompts (e.g. Open App, Sign Transaction).
Signer kit	Typed client for one device app's commands, built on the DMK.
Clear signing / blind signing	Clear signing shows human-readable details on the device; blind signing shows only a hash. Clear signing needs metadata, which the context module provides.
CAL	Crypto Assets List: Ledger's backend of token and asset metadata used for clear signing.
Speculos	Ledger's device emulator, for testing without hardware.
UTXO	Unspent Transaction Output: the Bitcoin-style accounting model, as opposed to Ethereum-style account balances.
PSBT	Partially Signed Bitcoin Transaction (BIP174 v0, BIP370 v2): the standard format the modern Bitcoin app signs.
Ledger Wallet / Ledger Live	Same product. Ledger Live was renamed Ledger Wallet; many repo and package names still say "live".
Live App / Wallet API	Web apps embedded in Ledger Wallet, and the RPC API they use to ask the host for accounts and signatures.
LKRP / Ledger Sync	Ledger Key Ring Protocol (historically "Trustchain"), which keeps wallet data end-to-end encrypted and synced across a user's Ledger Wallet installs.

Open questions worth checking

- Whether `device-signer-kit-bitcoin` (or the Bitcoin app's new protocol) supports Bitcoin-derived altcoins. The READMEs you shared only describe Bitcoin.
- What `device-contacts-kit`, `device-signer-kit-icp` and `device-signer-kit-xrp` do. They are in your notes but have no README in the material.
- Whether `DmkCompatTransport` is usable outside Ledger Wallet. It's in `live-dmk-shared`; I couldn't confirm from the material whether that package is published for external use.
- Official DMK documentation: the Ledger Developer Portal (developers.ledger.com) and the sample app linked from the DMK README.